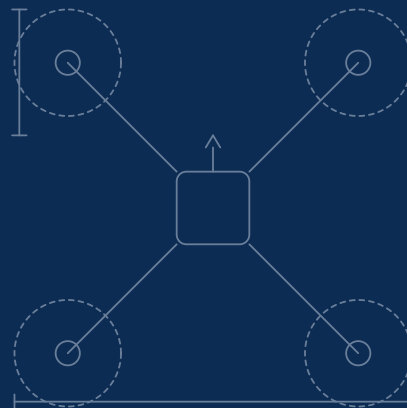


[FITS ANY ARDUPILOT / PX4 BUILD · BENCH-TEST WITH PROPS OFF · FLY LEGAL]

Six Upgrades Your Drone Is Missing

Add-ons for the drone you already own: an indoor hover with no GPS, a sensor ring that refuses to hit a wall, a parachute that saves it when it falls, a gimbal, a landing pad it finds by itself, and a payload that maps the invisible. Every one bolts onto a standard flight controller.



- | | | |
|-----------|---|-------------|
| 01 | Indoor Position Hold Rock-steady hover in a room with no GPS — a \$12 sensor that watches the floor | \$35 |
| 02 | Obstacle-Avoidance Sensor Ring A ring of ToF sensors that stops your drone before it hits the wall — and slides it away | \$50 |
| 03 | Parachute Recovery System A failsafe that senses freefall and pops a canopy — saves the drone, and whoever is below it | \$50 |
| 04 | 2-Axis Brushless Camera Gimbal Buttery footage from a shaky drone — FOC gimbal motors that cancel every wobble | \$60 |
| 05 | Vision Precision Landing It lands itself on a coaster-sized pad — a downward camera, an AprilTag, and a companion computer | \$55 |
| 06 | Signal-Mapping Payload Fly a grid and come back with a 3D heatmap of every radio signal in the air — a receive-only survey payload | \$50 |

MASS +142 G · CG +3 MM · UPGRADES 06 · PROPS OFF · FLY LEGAL

● By Enes · @1nska · Electrical Engineer — a Build List field guide



DRAWING Drone Upgrades		SHEET 1 of 1	
SCALE NTS	REV A	DATE 09.2026	
DRAWN BY @1nska		CHECKED Ohm's Club	

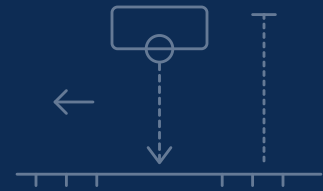
Indoor Position Hold

Rock-steady hover in a room with no GPS — a \$12 sensor that watches the floor

\$35

beginner

1 weekend



// why this matters

Outdoors, GPS tells the flight controller where the drone is. Indoors there is no GPS, and a drone drifts like a balloon in a draft. The fix is the same trick an optical mouse uses: a tiny downward camera (a PMW3901 optical-flow sensor) watches the floor texture slide by and reports velocity, while a rangefinder gives height. Feed both to the flight controller and it holds position — a hover you can let go of, indoors, that stays put. It is the cheapest, most dramatic upgrade there is, and it is the foundation for every kind of indoor autonomy.

WHAT THIS PROVES TO A RECRUITER

You understand optical flow (velocity from image motion), why flow must be scaled by height, how a flight controller's estimator fuses a new sensor, and how to configure and validate that fusion safely. That is the core of GPS-denied navigation.

→ **the job this maps to:** drone autonomy and GNC engineering — indoor-drone, inspection, and navigation teams.

! PROPS OFF FIRST

Configure and verify every new sensor with the propellers removed, watching the data in your ground station. Make the first hover low, in a large clear room, with a hand on the sticks and nobody else inside — and never over people. That rule applies to every upgrade in this guide.

// how it works

- ▶ **Optical flow:** The sensor sees floor texture move across its tiny image and reports pixel displacement per frame — the angular rate of the ground beneath. It is a mouse sensor pointed at the floor from a metre up.
- ▶ **Height turns flow into velocity:** The same angular rate means a faster ground speed when you are higher, so the flight controller needs the height from a rangefinder (a ToF like the VL53L1X or a TFmini) to scale flow into linear velocity.
- ▶ **Fusion in the estimator:** The flight controller's EKF fuses flow velocity with the IMU and the rangefinder into a position estimate whose drift is bounded by the flow — no GPS required. It also enables terrain-following altitude hold.
- ▶ **Light and texture:** Flow needs texture and light: a blank floor or a dim room starves it. A small downward LED and a textured surface make the difference between a locked hover and a drift.
- ▶ **Flight modes:** With flow enabled, position-hold and loiter modes work indoors, and the controller arbitrates between GPS and flow depending on what is healthy. Understanding that arbitration is what keeps a hybrid indoor / outdoor flight safe.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Optical-flow sensor	PMW3901 module	1	\$12
Rangefinder	VL53L1X or TFmini-S	1	\$14
Mount + wiring	printed bracket, cable	—	\$5
Downward LED (optional)	for dark floors	1	\$4

// build steps

1 Mount the sensors

Fix the flow sensor and rangefinder facing straight down, clear of props and landing gear. Alignment matters: a tilted flow sensor reads motion that isn't there.

2 Wire and enable

Connect flow (SPI or UART) and rangefinder (I2C or UART) to the flight controller, enable both in parameters, and confirm live readings in the ground station — props off.

3 Calibrate the flow scale

Move the drone by hand over a textured floor and compare flow-reported motion to the IMU; adjust the flow scale parameters until they agree.

4 Verify the estimator

With props still off, check that the EKF reports a healthy flow-based position estimate. Only then fit props.

5 Hover and let go

In a clear, well-lit room, take off low in position-hold and slowly release the sticks. Watch it stay put. Then test terrain-following over a step and note where flow breaks (dark floor, low texture).

// interview questions

- How does an optical-flow sensor estimate velocity?
- Why must optical flow be scaled by height above ground?
- How does a flight controller's EKF fuse flow, IMU, and rangefinder data?
- What conditions cause optical flow to fail?
- How does the controller arbitrate between GPS and flow?
- How would you validate a new sensor before flying with it?

[UPGRADE 02]

Obstacle-Avoidance Sensor Ring

A ring of ToF sensors that stops your drone before it hits the wall — and slides it away

\$50

intermediate

1-2 weekends



// why this matters

A drone that can feel the walls is a fundamentally safer drone. Mount a ring of time-of-flight distance sensors around the frame, have a small microcontroller stream the nearest obstacle in each direction to the flight controller as proximity data, and the controller's own avoidance logic takes over: fly toward a wall and the drone stops short, or slides along it, no matter what the sticks say. Push it toward a doorway and it refuses to hit the frame. It is the flagship of this guide — sensing, integration, and control behaviour in one upgrade — and it is the same layered safety architecture real autonomous drones use.

WHAT THIS PROVES TO A RECRUITER

You understand multi-sensor proximity sensing, sector-based obstacle representation, streaming sensor data to a flight controller over a standard protocol, and how avoidance behaviour (stop versus slide, safety margins) is designed and tuned. That is core drone-safety and autonomy engineering.

→ **the job this maps to:** drone safety and autonomy engineering — inspection-drone, delivery, and consumer-drone perception teams.

! A SAFETY LAYER, NOT A PILOT

Avoidance has blind spots (between sensors, above, below) and hard limits (glass, bright sun, speed). Test it slowly, low, in a clear space with a spotter, and never treat it as permission to fly near people. It catches mistakes; it does not replace the pilot.

// how it works

- ▶ **Sectors, not a single beam:** One ultrasonic sensor sees one direction. A ring of ToF sensors reports the nearest obstacle in each sector around the drone, giving the controller a 360° picture of the horizontal clearance.
- ▶ **Speaking the controller's language:** A microcontroller reads every sensor and packs the distances into a standard proximity message over the telemetry port. The flight controller ingests it exactly as it would a commercial sensor — it just works.
- ▶ **Stop or slide:** With avoidance enabled, the controller applies a margin: approach an obstacle and it either halts the drone at the margin or lets it slide along the obstacle. Both are tunable, and the choice changes how the drone feels to fly.
- ▶ **Field of view and gaps:** Each sensor covers a cone; between cones there are gaps, and nothing covers above or below. Sensor count, spacing, and mounting angle set how complete the picture is.
- ▶ **What ToF can't see:** Glass, mirrors, very dark surfaces, and bright sunlight fool infrared ToF. Speed matters too — at high speed the drone can outrun its own reaction distance. Knowing the limits is part of the design.

BUILD NOTE

Six VL53L1X modules at 60° spacing is the cheap version; four TFmini-S give longer range. Either way, the microcontroller sends standard MAVLink OBSTACLE_DISTANCE messages, so the flight controller's built-in avoidance does the hard part — you don't write the control loop.

// key design decisions

- **Why a ring of ToF sensors instead of one ultrasonic sensor?** A single sensor sees one narrow direction and ultrasonics are slow and wide-beamed. A ring of fast, narrow ToF sensors gives clearance in every horizontal direction, updated many times a second — the minimum for the drone to react to a wall it is drifting toward, not just one it is facing.
- **Why send proximity to the flight controller instead of overriding the sticks yourself?** The flight controller already has the state estimate, the motor mixing, and a tested avoidance algorithm with proper margins. Feeding it standard proximity data means your sensors plug into that safety logic instead of fighting it with a home-made override that could destabilise the aircraft.
- **Why is this not autonomy?** Avoidance is a reflex: it prevents contact within its field of view at modest speeds. It has gaps, it can be fooled, and it plans nothing. Real autonomy adds mapping, planning, and redundancy on top. Treating avoidance as a safety net rather than a self-flying feature is the honest engineering position.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
ToF sensors (×6)	VL53L1X (or 4× TFmini-S)	6	\$36
Microcontroller	ESP32 / Arduino as sensor bridge	1	\$8
Ring mount	printed ring / brackets	1	\$5
Wiring	cables, connectors	—	\$2

// build steps

1 Build the ring

Mount six ToF sensors at 60° spacing on a printed ring around the frame, facing outward and level, clear of props. Wire them to the microcontroller.

2 Read every sector

Program the microcontroller to poll all sensors and print the six distances. Wave a hand around the drone and confirm each sector responds.

3 Stream to the flight controller

Send the distances as MAVLink OBSTACLE_DISTANCE messages over a telemetry port. Confirm the ground station's proximity view shows your sectors live — props off.

4 Enable avoidance and set margins

Turn on the controller's avoidance, set a conservative margin (a metre or more) and choose stop or slide. Hover in a clear space and slowly push toward a wall: it should stop short.

5 Tune and map the limits

Adjust margins and behaviour, test corners and doorways, then deliberately find the failure modes — glass, sun, speed — and document them. Knowing where it fails is the deliverable.

// interview questions

- Why does obstacle avoidance need coverage in all directions?
- How do you send proximity data to a flight controller?
- What is the difference between stop and slide avoidance behaviour?
- What limits do infrared time-of-flight sensors have?
- How do sensor count and spacing affect coverage gaps?
- Why is avoidance a safety layer rather than autonomy?

Parachute Recovery System

A failsafe that senses freefall and pops a canopy — saves the drone, and whoever is below it

\$50

intermediate

1 weekend



// why this matters

Motors fail, batteries fail, and a drone that loses power becomes a falling brick. A parachute recovery system detects the fall — or takes a command — kills the motors, and deploys a canopy so the aircraft comes down at a walking pace instead of terminal velocity. Commercial systems cost hundreds; you can build one with a spring-loaded tube, a servo pin, an accelerometer, and a properly sized canopy. It is the one upgrade that protects people on the ground as much as the drone, it is increasingly what regulators want for flights near people, and the deployment shot is spectacular.

WHAT THIS PROVES TO A RECRUITER

You understand freefall detection from an IMU, failsafe logic and false-trigger rejection, mechanical deployer design, canopy sizing from descent-rate physics, and the test discipline (bench, drop, tethered, live) that a safety system demands. Safety-critical design is a skill every aviation employer looks for.

→ **the job this maps to:** drone safety-systems and aviation-safety engineering — delivery-drone, UAS-operations, and airworthiness teams.

! TEST IT LIKE IT MATTERS

A deployer is a spring or a gas charge under tension: keep hands clear of the tube, test with props off, and never stand under a drop test. Use spring or servo release — no pyrotechnics. Do live deploy tests only over an empty field, well within your local rules, and remember a canopy in the props is worse than no canopy: motor cut-off on deploy is mandatory.

// how it works

- ▶ **Detecting the fall:** In freefall the accelerometer reads near zero g on all axes. Detect that condition for longer than a set time (a few hundred milliseconds, to reject bumps) and you have a reliable trigger. A failsafe signal from the flight controller, or a manual channel, are the other triggers.
- ▶ **Cut the motors:** On deploy, the props must stop instantly, or the canopy and lines wrap into them. The trigger sends a motor-kill to the flight controller (or cuts ESC power) before the pin releases.
- ▶ **The deployer:** A tube holds the packed canopy on a compressed spring, retained by a pin a servo pulls. Release the pin and the spring throws the canopy clear of the airframe; a bridle and swivel connect it to the frame's centre.
- ▶ **Sizing the canopy:** Descent rate depends on canopy area and total mass; aim for roughly four to five metres per second. Too small and the landing is a crash; too large and the pack won't fit. The maths is simple drag physics.
- ▶ **Testing a safety system:** Bench trigger tests, then drop tests with a dummy mass from height, then tethered deploy, then a live test over an empty field. A parachute you haven't drop-tested is decoration.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Canopy	drone parachute sized for your mass	1	\$25
Deployer	printed tube, spring, servo + pin	1	\$12
Trigger	IMU + microcontroller (or FC channel)	1	\$8
Bridle + mount	swivel, cord, frame mount	—	\$5

// build steps

1 Size the canopy

Weigh your drone, choose a target descent rate, and compute the canopy area. Buy or sew a canopy to match and pack it with a spring-loaded deployer tube.

2 Build the deployer

Print the tube, fit the spring and a servo-pulled retaining pin, and mount it at the frame's centre with a bridle and swivel. Confirm a clean, fast throw on the bench — props off.

3 Write the trigger

Detect near-zero g for a few hundred milliseconds on the IMU, plus a manual trigger channel. Wire a motor-kill to the flight controller that fires before the pin releases.

4 Drop-test with a dummy

Drop the deployer with a dummy mass from height over soft ground, many times. Measure the descent rate and the deploy delay; tune spring, packing, and trigger time.

5 Tethered, then live

Fit it to the drone, verify motor kill on trigger with props off, then do a tethered deploy and finally a live test over an empty field at safe altitude. Film the deploy — and log the descent rate.

// interview questions

- How can an IMU detect that a drone is in freefall?
- Why must the motors be cut before a parachute deploys?
- How do you size a parachute for a given descent rate?
- How do you reject false triggers in a freefall detector?
- What is the correct test sequence for a safety-critical deployer?
- Why are parachutes becoming a regulatory expectation for drones?

2-Axis Brushless Camera Gimbal

Buttery footage from a shaky drone — FOC gimbal motors that cancel every wobble

\$60

advanced

1–2 weekends



// why this matters

Raw drone footage is a jittery mess because the camera rides every twitch of the airframe. A brushless gimbal fixes that: an IMU on the camera measures its attitude hundreds of times a second and two brushless motors, driven smoothly with field-oriented control, hold it level no matter what the drone does. Build one and your footage goes from home video to broadcast-smooth. It is mechatronics, control, and the same FOC drive technology behind robot joints — and it is the upgrade that most changes what your drone produces.

WHAT THIS PROVES TO A RECRUITER

You understand gimbal mechanics and balancing, IMU-based attitude sensing on a moving base, field-oriented control of brushless motors for smooth torque, PID tuning of a stabilisation loop, and vibration isolation. Stabilisation engineering shows up across cameras, drones, weapons-sights, and satellites.

→ **the job this maps to:** stabilisation and mechatronics engineering — camera-gimbal, drone-payload, and precision-pointing teams.

// how it works

- ▶ **Sensing the camera's attitude:** An IMU mounted on the camera plate reports its pitch and roll at high rate. The controller's job is to hold that attitude at the target regardless of what the frame beneath it does.
- ▶ **Gimbal motors and FOC:** Gimbal brushless motors are high-pole-count, low-speed motors driven with field-oriented control for smooth, cogging-free torque at zero speed — exactly what holding a camera still requires.
- ▶ **The stabilisation loop:** A PID loop per axis takes the attitude error and commands motor torque to cancel it. Tuning sets the stiffness and damping: too soft and it wobbles, too stiff and it oscillates.
- ▶ **Balance is everything:** If the camera's centre of mass sits off the motor axes, the motors fight gravity constantly, wasting torque and ruining smoothness. Mechanical balance on each axis before any tuning is the golden rule.
- ▶ **Isolating vibration:** Motor and prop vibration reaches the gimbal through the frame; rubber dampers between drone and gimbal filter it so the IMU sees camera motion, not airframe buzz.

BUILD NOTE

Don't write gimbal firmware from scratch. A STorM32 or SimpleBGC-class controller with its IMU handles FOC and stabilisation; your engineering is the frame, the balance, the motor choice, the tuning, and the vibration mounting. Balance first — an unbalanced gimbal can never be tuned smooth.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Gimbal motors (×2)	brushless gimbal motors, sized for your camera	2	\$24
Gimbal controller + IMU	STorM32 / SimpleBGC-class board	1	\$25
Frame	3D-printed 2-axis frame	1	\$8
Dampers + wiring	rubber damper balls, cables	—	\$3

// build steps

1 Print and assemble the frame

Build the two-axis frame with pitch and roll motors, and mount your camera plate.

2 Balance each axis

Adjust the camera position until it stays put at any angle with the motors unpowered. Do this before anything else.

3 Wire and configure

Connect motors, IMU, and controller; set motor poles, IMU orientation, and directions in the configurator. Confirm the camera levels when powered.

4 Tune the loops

Raise P until it holds firmly, add D to stop oscillation, add I to remove droop. Tilt the base by hand and watch the camera stay level.

5 Mount, damp, and fly

Fit it to the drone on rubber dampers, check clearance from props and landing gear, and fly. Compare footage with and without stabilisation — the before / after is the video.

// interview questions

- Why must a gimbal be mechanically balanced before tuning?
- Why are brushless gimbal motors driven with field-oriented control?
- What does each PID term do in a stabilisation loop?
- Where should the IMU be mounted on a gimbal, and why?
- How does vibration reach the gimbal, and how is it isolated?
- How would you add a third (yaw) axis?

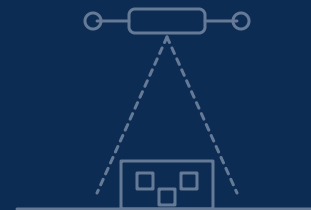
Vision Precision Landing

It lands itself on a coaster-sized pad — a downward camera, an AprilTag, and a companion computer

\$55

advanced

2 weekends



// why this matters

GPS puts a drone down within a couple of metres, which is useless for docking, charging, or landing on a vehicle. Precision landing uses a downward camera to see a fiducial marker on the pad, computes exactly where the pad is relative to the drone, and steers the descent so it touches down centred — the technology behind every autonomous docking and charging drone. Add a small companion computer, a camera, and a printed tag, and your drone lands itself on a pad the size of a coaster. It is computer vision, coordinate transforms, and controller integration, and the demo is undeniable.

WHAT THIS PROVES TO A RECRUITER

You understand fiducial detection and pose estimation, camera calibration, sending a landing target to a flight controller over a standard protocol, the controller's precision-landing behaviour, and failure handling when the target is lost. That is the vision-to-control pipeline autonomous drones are built on.

→ **the job this maps to:** drone autonomy and computer-vision engineering — docking, delivery, and autonomous-inspection teams.

! CLEAR PAD, EMPTY FIELD

Test precision landings over an empty field with a clear area around the pad and an abort ready on the sticks. Start from low altitude and increase as it proves reliable. A drone descending onto a target is committed — make sure nothing can be under it.

// how it works

- ▶ **Fiducials:** An AprilTag or ArUco marker is a black-and-white pattern designed for reliable detection; from its corners in the image, the software recovers the tag's position and orientation relative to the camera.
- ▶ **The companion computer:** A small computer (a Raspberry Pi) runs the camera and the tag detector, then sends the target's offset to the flight controller as a standard landing-target message over the telemetry link.
- ▶ **The controller lands on it:** In precision-landing mode, the flight controller uses the target offset to steer the descent, correcting horizontally as it comes down so the drone stays centred over the pad.
- ▶ **Calibration and alignment:** Camera calibration turns pixels into angles correctly; camera-to-body alignment turns those into offsets the controller can act on. Get either wrong and it lands confidently next to the pad.
- ▶ **Losing the target:** Tags disappear when the drone is too high, too tilted, or too close. Robust behaviour handles that: keep the last estimate briefly, then fall back to a normal landing or abort.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Companion computer	Raspberry Pi Zero 2 / Pi 4	1	\$30
Camera	Pi camera, downward	1	\$10
Tag + pad	printed AprilTag on a board	1	free
Mount + wiring + power	bracket, telemetry cable, BEC	—	\$15

// build steps

1 Calibrate the camera

Mount the camera facing down and run a calibration so the detector reports accurate angles.

2 Detect the tag on the bench

Run the AprilTag detector and confirm it reports the tag's offset and distance as you move the tag under the camera.

3 Talk to the flight controller

Wire the companion computer to a telemetry port and send landing-target messages. Confirm the controller reports a valid target — props off.

4 Low-altitude landings

Over an empty field, take off, enable precision landing from a couple of metres, and watch it correct and touch down on the pad. Measure the error.

5 Increase altitude and harden

Land from higher up, test in wind, and add the lost-target fallback. Log the landing error over many attempts — that consistency is your result.

// interview questions

- How does a fiducial marker give both position and orientation?
- What is a companion computer's role in a drone system?
- How does the flight controller use a landing target during descent?
- Why do camera calibration and alignment matter so much here?
- What should happen when the target is lost mid-descent?
- How would you extend this to landing on a moving vehicle?

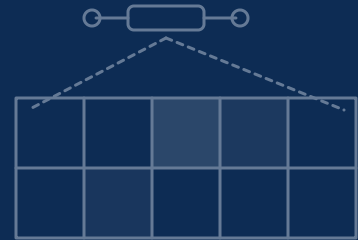
Signal-Mapping Payload

Fly a grid and come back with a 3D heatmap of every radio signal in the air — a receive-only survey payload

\$50

advanced

1-2 weekends



// why this matters

A drone is the perfect platform for measuring things you can't see from the ground — and radio is the best example. Strap on a receive-only payload (an SDR or a WiFi scanner) with a GPS logger, fly a lawnmower grid at a few altitudes, and you come back with a three-dimensional heatmap of signal strength: where your WiFi dies, how far your mesh radios reach, where interference lives. It is the drone as a scientific instrument, it is receive-only, and it produces the one thing nobody else on your feed has: a picture of the invisible over your own neighbourhood.

WHAT THIS PROVES TO A RECRUITER

You understand payload integration (mass, power, isolation from the drone's own RF noise), geotagged data logging, survey flight planning, spatial interpolation into a heatmap, and interpreting RF propagation. Aerial survey and sensing payloads are a growing engineering field.

→ the job this maps to: drone-payload and aerial-survey engineering — inspection, mapping, and RF-survey teams.

! RECEIVE-ONLY, YOUR OWN AIRSPACE

The payload only listens — it transmits nothing beyond your drone's normal control link. Fly the survey within your local rules: line of sight, altitude limits, not over people or others' property, and keep the added mass well inside your drone's limits.

// how it works

- ▶ **The payload:** A lightweight receiver (WiFi scanner or SDR), a GPS logger, and a battery, mounted away from the drone's own antenna so its transmitter doesn't swamp the measurement.
- ▶ **Geotagged logging:** Every measurement is stamped with position, altitude, and time, so each reading is a point in 3D space. Logging rate sets your spatial resolution.
- ▶ **The survey pattern:** A planned lawnmower grid at constant altitude covers the area evenly; repeating it at two or three altitudes builds the third dimension. Mission planning makes it repeatable.
- ▶ **From points to a heatmap:** Interpolating scattered readings (inverse-distance or kriging) turns points into a smooth map; stacking altitudes gives a 3D volume you can slice.
- ▶ **Reading the map:** Signal strength varies with distance, obstacles, multipath, and your payload antenna's own pattern. Interpreting the map honestly — separating real coverage from artefacts — is the engineering.

BUILD NOTE

Start with an ESP32 WiFi scanner (cheapest, lightest, logs every access point's strength) before an SDR. Isolate the payload's antenna from the drone's own transmitter, and always check hover time with the payload aboard — extra mass shortens flights faster than you expect. Swap in a small thermal sensor and the same pipeline maps roof heat loss.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Receiver	ESP32 WiFi scanner (or RTL-SDR)	1	\$8–25
GPS logger	GNSS module + microSD logging	1	\$12
Battery + power	small LiPo, regulator	1	\$5
Mount + antenna	printed payload mount, antenna	—	\$8

// build steps

1 Build and bench-test the payload

Assemble receiver, GPS, and logger; confirm it logs signal strength with position and time while you walk around.

2 Mount it and check the aircraft

Fit the payload away from the drone's antenna, check centre of gravity, and hover-test to confirm flight time and stability with the extra mass.

3 Plan the survey

Define a lawnmower grid over your area at a fixed altitude in the mission planner, within line of sight and altitude limits.

4 Fly and log

Fly the grid (autonomously or by hand), then repeat at one or two more altitudes. Land and pull the logs.

5 Build the heatmap

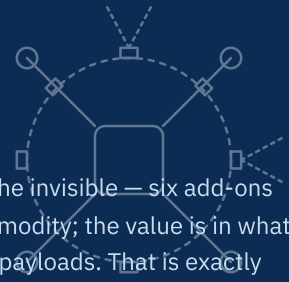
Interpolate the readings into a map per altitude, stack them into a 3D view, and annotate what you find — dead zones, strong spots, interference. Publish it.

// interview questions

- How do you keep a receiver payload from being swamped by the drone's own transmitter?
- Why must every measurement be geotagged with altitude, not just position?
- How do you plan a survey pattern for even coverage?
- How do you turn scattered readings into a heatmap?
- What artefacts can mislead an RF survey?
- How does added payload mass affect flight performance and safety?

[WRAP]

Your drone is a platform — the upgrades are the engineering.



An indoor hover, a wall it refuses to hit, a parachute, a gimbal, a pad it lands on by itself, a map of the invisible — six add-ons that turn a stock quad into a sensing, self-protecting, self-landing instrument. The airframe is commodity; the value is in what you bolt on and integrate: sensor fusion, avoidance logic, safety systems, stabilisation, vision, and payloads. That is exactly the work drone and autonomy companies hire for, and it is far more impressive than a kit build.

// the skill ladder

Fit them in order and each one hands you the technique the next one needs:

sensing → avoidance → safety systems → stabilisation → vision autonomy → payloads

! BEFORE YOU FLY

Know your local rules — registration, line of sight, altitude limits, no flying over people or near airports, lights at night — and follow them. Bench-test every upgrade with the props off and verify the data before a single flight. Check mass and centre of gravity after each add-on, keep failsafes configured, respect LiPo safety, and test new behaviours low, slow, in an empty field, with a spotter. A drone is an aircraft; treat every upgrade like an airworthiness change.

DO THIS

Fit even one. Film it — a hover you let go of, a drone stopping short of a wall, a canopy popping mid-air, footage going from shaky to glass-smooth, a touchdown centred on a coaster, a heatmap of your street — and put the config, code, and test logs on GitHub with an honest note on what failed. One upgrade and one clean repo prove you can engineer an aircraft, not just assemble one.

Full build notes, parts links, and the next field guide go out in **The Build List**. The people actually shipping these — and trading tuning and payload tricks — are in **Ohm's Club**.

— Enes (@1nska) · Electrical Engineer